

Plan to Add an Ex-situ Satellite-NSC Channel

Updated against the latest main_spatial.py and evo.py code structure

0. Executive summary

The present implementation does not explicitly inject ex-situ GCs at their MPB merger time and does not assign them a physical satellite velocity. Instead, the formation stage creates GCs along all retained tree branches, assigns a scalar radius, and writes a formation catalog. The evolution stage then evolves every input row with the same analytical MPB-background solver. The most local and low-risk update is therefore to build synthetic ex-situ NSC rows before evo.py is called, using the same catalog interface evo.py already understands.

Recommended first implementation: create a catalog-level preprocessor or a small module called from main_spatial.py. For each non-MPB branch, decide which satellite GCs sink to the satellite center before branch release, combine those sunk components into one GC-like ex-situ NSC object, write that object into an augmented all catalog at the release redshift, and then let evo.py evolve it exactly like a normal massive GC.

1. Corrected reading of the current code

The table below replaces the earlier vague description with code-level behavior.

Topic	Current implementation	Implication
Where GCs form	main_spatial.py walks all retained tree rows and triggers GC formation when the halo growth-rate criterion is satisfied.	GCs are born on MPB and non-MPB branches at their formation redshifts.
MPB flag	The formation loop computes <code>is_mpb = (mpi[i] == mpbi)</code> and stores it in the in-memory GC object.	The branch origin is known during formation.
Radii at birth	MPB GCs are assigned a Sersic-sampled galactocentric radius. Non-MPB GCs are assigned <code>fallback_outer_kpc = 0.5 Rvir / 1000</code> .	The current ex-situ radius is a scalar outer-halo proxy, not a satellite orbital radius.
Velocity	The GC object and output catalog do not store velocity. evo.py computes <code>vc</code> internally from the analytical background.	There is no inherited satellite velocity in the current pipeline.
Catalog used by evo.py	The modern 13-column formation catalog includes halo ID, host mass, mass, zform, metallicity, radius, cluster size, surface density, and IMBH mass.	The all catalog does not carry <code>isMPB</code> , so evo.py cannot distinguish in-situ, loose ex-situ, or ex-situ NSC rows.
Background used by evo.py	evo.py reads only the monotonic MPB-like block from the halo-evolution table.	All input rows are evolved in the MPB analytical potential, not in their original satellite potential.
Evolution terms	evo.py applies stellar-evolution mass loss, tidal stripping, DF radial decay, disruption, central sinking, and IMBH-wanderer handling.	An ex-situ NSC can be evolved by evo.py if it is encoded as a normal GC-like input row.

2. Target behavior after the update

The desired model should be:

```
satellite GCs form on non-MPB branches
-> some satellite GCs sink to the satellite center before release
-> those sunk GCs are merged into one satellite NSC object
-> the satellite NSC is released into the MPB as a GC-like object
-> evo.py evolves this object with the normal GC solver
-> it may survive, disrupt, sink to the center, or become an IMBH wanderer
```

This keeps the final MPB contribution non-forced. The ex-situ NSC is not directly added to the MPB center. It is only inserted into the MPB GC list, and the existing evolution code decides its fate.

3. Minimal data model changes

For the first version, avoid changing evo.py. Add metadata during formation and compress it back into a 13-column evo-compatible row before evolution.

Field	Where to store first	Purpose
branch_id	GC object in main_spatial.py	Group GCs by satellite branch. Use mpi or another stable branch label.
object_type	Sidecar file; optional catalog extension later	0 = in-situ GC, 1 = loose ex-situ GC, 2 = ex-situ satellite NSC.
local_rGalaxy	GC object	Radius inside the original host branch; needed for satellite sinking.
release_redshift	Sidecar and synthetic row	The redshift at which the satellite product is injected into the MPB catalog.
component_gc_ids	Sidecar only	Audit trail for which GCs were merged into each synthetic NSC.
nsc_imbh_mass	Synthetic row column 13	Sum of the component IMBH masses. Do not reseed a new IMBH for the NSC.

4. Where to implement

File	Change needed	Reason
main_spatial.py	Add branch metadata, construct satellite NSCs, write an augmented formation catalog, and write a sidecar audit table.	This file already knows the tree, branch identity, formation time, GC masses, metallicity, radii, and IMBH masses.
evo.py	No required change for the first implementation.	It already evolves any catalog row as a GC-like object if mass, zform, radius, and IMBH mass are supplied.
IMBH.py	No change.	The synthetic NSC should inherit summed IMBH mass from component GCs, not generate a new one.
new_exsitu_nsc.py	Optional but recommended.	Keeps branch-release inference and NSC construction out of legacy-style main_spatial.py.

5. Branch release time

The code currently loads tree columns `m`, `fp`, `subid`, `mpi`, `redshift`, and `spin norm`. It does not load coordinates or velocities. Therefore, for the first version, define the branch release time from the retained tree structure rather than from satellite phase space.

Fiducial release definition:

```
branch_rows = np.where(mpi == branch_id)[0]
z_release = np.min(redshifts[branch_rows])
t_release = cosmicTime(z_release)
```

Interpretation: the last retained row of a non-MPB branch is the time when the branch is no longer followed independently in the fixed tree. This is the simplest available proxy for satellite release into the MPB. If the inferred release time is later than the requested final redshift, do not inject that branch into the MPB for that run.

Implementation warning:

The current `loadTree` function filters rows by `final_redshift` before returning arrays. For a reliable branch-release map, build the release map from the full untruncated tree first, then apply `final_redshift` filtering to the formation history. Otherwise high- z runs may incorrectly treat still-unreleased side branches as available to the MPB.

6. Local satellite radius for non-MPB GCs

The present non-MPB radius is $0.5 R_{\text{vir}}$ of the local host at formation. That is useful as an outer-halo release proxy, but it is not the correct radius for deciding whether a GC sinks to the satellite nucleus before release. For the satellite-NSC channel, use two radii:

- **local_rGalaxy**: the GC radius inside its own branch at formation. This should be Sersic-sampled from the satellite host, using the same sampling logic as MPB GCs.
- **rGalaxy**: the MPB radius written to the final catalog. For released loose ex-situ GCs and synthetic NSCs, this becomes the MPB release radius.

Recommended first version:

```
if gc.is_mpb:
    gc.local_rGalaxy = sersic_sample(host=MPB_or_branch)
    gc.rGalaxy = gc.local_rGalaxy
else:
    gc.local_rGalaxy = sersic_sample(host=satellite_branch)
    gc.rGalaxy = release_radius_in_MPB # set later, at release
```

7. Satellite sinking criterion

For each GC in a non-MPB branch, estimate whether it reaches the satellite center before branch release. The easiest consistent criterion is to reuse the same scaling as `evo.py` dynamical friction, where the radial decay rate is proportional to $M / (0.45 r v_c)$.

```
t_form = cosmicTime(gc.origin_redshift)
t_df_sat = estimate_df_time_in_satellite(gc.mass, gc.local_rGalaxy,
                                         satellite_mass, satellite_spin,
                                         t_form)

if t_form + t_df_sat < t_release:
    gc.status_pre_release = "component_of_satellite_NSC"
else:
    gc.status_pre_release = "loose_exsitu_GC"
```

A simple first estimator is $t_{\text{df_sat}} \approx 0.45 r_{\text{local}}^2 v_{\text{c_sat}} / M_{\text{gc}}$, in the same unit convention used by `evo.py`. The circular velocity can be estimated from the satellite host mass and local radius, either by importing `evo.py` helper functions or by using a virial proxy for the first diagnostic run.

8. Pre-release evolution

Before release into the MPB, satellite GCs should not be evolved in the MPB potential. For a low-risk first implementation, do only approximate pre-release mass evolution inside `main_spatial.py`, then start the released objects in `evo.py` at `z_release`.

Object before release	Action before release	Catalog row written to <code>evo.py</code>
Loose ex-situ GC	Apply stellar-evolution mass loss and optionally weak tidal disruption from <code>zform</code> to <code>z_release</code> .	One normal GC-like row with mass at release, <code>zform = z_release</code> , <code>rGalaxy = r_release</code> .
GC that sinks in satellite	Evolve until <code>t_sink</code> or <code>t_release</code> , then remove from loose population.	No individual row; it becomes part of a synthetic NSC.
Synthetic satellite NSC	Sum the remaining stellar masses and IMBH masses of sunk components; optionally evolve as a central satellite cluster until release.	One GC-like row with mass = <code>M_NSC</code> , IMBH mass = <code>sum(IMBH)</code> , <code>zform = z_release</code> , <code>rGalaxy = r_release</code> .

9. MPB release radius and velocity

The current code does not load positions or velocities from the tree. Therefore the first implementation should not pretend to use a physical ex-situ velocity. `evo.py` does not consume velocity anyway; it computes a circular speed from the analytical background. Use a scalar release radius only.

Fiducial radius:

```
r_release = f_release * Rvir_MPB(z_release)
f_release = 0.5 # fiducial
# bracket: f_release = 0.2, 0.5, 1.0
```

This preserves continuity with the current outer-halo placement while moving the timing from `zform` to `z_release`. Later, if the tree is extended to load subhalo coordinates and velocities, `r_release` and `v_release` can be replaced by the satellite-MPB relative phase-space values. That is a second-stage upgrade, not required for the present plan.

10. Output catalogs

Keep the original outputs for comparison. Add an augmented catalog and a sidecar audit table.

Output	Format	Use
<code>all_<Ns>.txt</code>	Current 13-column catalog	Baseline current model. Keep unchanged for reproducibility.
<code>all_exsitu_nsc_<Ns>.txt</code>	Same first 13 columns as <code>all_<Ns>.txt</code>	Input to <code>evo.py</code> for the upgraded model. Contains in-situ GCs, released loose ex-situ GCs, and synthetic ex-situ NSCs.
<code>nsc_components_<Ns>.txt</code>	Sidecar table with IDs and metadata	Records which GCs were absorbed into each satellite NSC, release redshift, component masses, and IMBH sums.
<code>branch_release_<Ns>.txt</code>	Sidecar table	Records <code>branch_id</code> , release redshift, release time, MPB release radius, and number of released objects.

11. Pseudocode

```
# After clusters have been formed for one z=0 host in main_spatial.py:

branch_release = build_branch_release_map(full_tree, mpbi, final_redshift)
augmented_rows = []
sidecar_rows = []

for branch_id in all_branch_ids:
    branch_gcs = [gc for gc in clusters if gc.branch_id == branch_id]

    if branch_id == mpbi:
        for gc in branch_gcs:
            gc.rGalaxy = gc.local_rGalaxy
            augmented_rows.append(make_catalog_row(gc,
                mass=gc.mass, z_out=gc.origin_redshift,
                r_out=gc.rGalaxy, object_type=0))
        continue

    release = branch_release[branch_id]
    if release.t_release > t_final:
        write_unreleased_sidecar(branch_id, branch_gcs)
        continue

    sunk_components = []
    loose_components = []

    for gc in branch_gcs:
        t_form = cosmicTime(gc.origin_redshift)
        t_df_sat = estimate_df_time_in_satellite(gc, release)

        if t_form + t_df_sat < release.t_release:
            sunk_components.append(gc)
        else:
            loose_components.append(gc)
```

```

for gc in loose_components:
    m_rel = pre_evolve_gc_mass(gc, release.t_release)
    if m_rel <= 0:
        continue
    augmented_rows.append(make_catalog_row(gc,
        mass=m_rel, z_out=release.z_release,
        r_out=release.r_release_mpb, object_type=1))

if len(sunk_components) > 0:
    nsc = build_synthetic_nsc(sunk_components, release)
    if nsc.mass > 0:
        augmented_rows.append(make_catalog_row(nsc,
            mass=nsc.mass, z_out=release.z_release,
            r_out=release.r_release_mpb,
            imbh_mass=nsc.imbh_mass, object_type=2))
        sidecar_rows.extend(component_audit_rows(nsc, sunk_components))

write_augmented_all_catalog(augmented_rows)
write_nsc_sidecar(sidecar_rows)

```

12. Mass and BH bookkeeping

- Do not double count sunk components. A GC that becomes part of a satellite NSC must not also be written as a loose ex-situ GC row.
- Do not reseed the synthetic NSC with IMBH.py. The NSC IMBH mass is the sum of component IMBH masses.
- The synthetic NSC mass should include the stellar bound mass plus the carried IMBH mass only once. Keep the existing evo.py convention that `m_gc` is the total bound object mass and `m_imbh` is a protected lower bound.
- If the NSC stellar mass is stripped in evo.py until only IMBH remains, the existing wanderer logic can naturally take over.

13. Diagnostics to add before science interpretation

Diagnostic	Check
Mass conservation	For each branch: initial GC mass approximately equals loose released mass + NSC released mass + pre-release losses + unreleased/disrupted mass.
No double counting	Every original non-MPB GC ID appears in exactly one category: loose, NSC component, disrupted before release, or unreleased.
Release timing	Histogram of <code>zform</code> , <code>zrelease</code> , and <code>final_z</code> . No object should be injected before it forms or after the target epoch.
Contribution split	Plot MPB-center mass from in-situ GCs, loose ex-situ GCs, and ex-situ NSC objects separately.
Radius sanity	Compare current all_<Ns>.txt radii with upgraded release radii. Verify that release radii are not accidentally used as satellite local radii.
BH sanity	Compare total IMBH mass formed, retained in loose GCs, carried by ex-situ NSCs, sunk to center, and surviving as wanderers.

14. Recommended model variants

Variant	Description	Purpose
Current	Original all_<Ns>.txt into evo.py.	Baseline.
Release-only	Non-MPB GCs are injected at z_release instead of zform, but no satellite NSC merging is applied.	Isolate the effect of correcting the timing.
NSC-fiducial	Satellite-sunk GCs are merged into synthetic NSCs and injected at z_release with f_release = 0.5.	Main upgraded model.
NSC-radius bracket	Same as fiducial but f_release = 0.2, 0.5, 1.0.	Estimate uncertainty from unknown satellite phase space.
NSC-local-DF bracket	Vary satellite sinking criterion by factors of 0.3, 1, and 3 in t_df_sat.	Estimate uncertainty from approximate pre-release satellite evolution.

15. What not to do in the first version

- Do not directly add satellite NSC mass to the MPB central NSC at release. That would bypass evo.py and force the answer.
- Do not invent satellite velocities. The current tree parser and evo.py interface do not support them.
- Do not overwrite all_<Ns>.txt. Keep the current catalog and write an upgraded catalog with a new filename.
- Do not modify the IMBH seeding model for synthetic NSCs. Summed component IMBH mass is enough for the first implementation.

16. Limitations of the first version

- The release radius is still a scalar proxy. It is better timed than the current zform placement, but it is not a true satellite orbit.
- Pre-release evolution is approximate unless a later version runs evo.py separately in each satellite branch before release.
- Branch release inferred from the last retained branch row should be checked manually for several example trees.
- The augmented catalog can encode object type only through a sidecar table unless evo.py is extended to read optional extra columns.

17. Final recommended implementation order

- Add branch_id, object_type, and local_rGalaxy to the in-memory GC object in main_spatial.py.
- Build a release map for every non-MPB branch from the full tree before final-redshift truncation.
- Change non-MPB local radii to satellite Sersic sampling; reserve 0.5 Rvir only for MPB release radius.
- Classify non-MPB GCs into loose and satellite-sunk components using the satellite DF criterion.
- Build one synthetic ex-situ NSC per branch if the branch has sunk components.
- Write all_exsitu_nsc_<Ns>.txt using the same first 13 columns expected by evo.py.
- Write nsc_components_<Ns>.txt and branch_release_<Ns>.txt as sidecar diagnostics.
- Run evo.py unchanged on both all_<Ns>.txt and all_exsitu_nsc_<Ns>.txt.
- Compare center mass, deposited mass profile, final GC mass, IMBH wanderers, and status fractions between models.

Bottom line

The best first upgrade is catalog-level: convert part of the non-MPB GC population into synthetic satellite NSC objects at branch release, then pass those objects into evo.py as ordinary massive GC-like rows. This corrects the missing satellite-NSC channel while preserving the existing analytical GC-evolution machinery.